

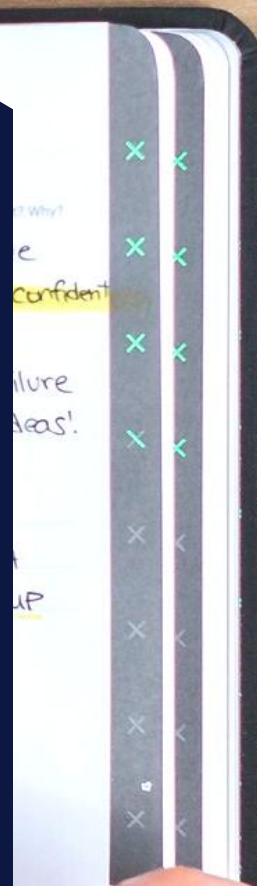


Práctica # 04

Patrones de Diseño de
APIs

Elaborado por:

David Oswaldo Aristega Salinas



Actividades

Práctica: 04 – GraphQL: Optimización de Modelos de Datos en Proveedores de Servicios con DataLoader

Asignatura: Patrones de Diseño de APIs · **Carrera:** Maestría de Software · **Docente:** Ing. Jennifer Yépez A., MSc.

Descripción de la actividad

1. Contexto del problema

La empresa ficticia **LogiTech SaaS** necesita un tablero unificado que muestre sus clientes junto con sus facturas históricas. Con una API REST, el frontend debe hacer 1 petición a `/clientes` y luego N peticiones a `/clientes/{id}/facturas`, una por cliente. Este exceso de viajes se conoce como **under-fetching** y satura las redes móviles.

GraphQL resuelve el problema del cliente: toda la información se pide en **una sola petición HTTP**. Sin embargo, si los resolvers no se diseñan con cuidado, el problema se traslada al servidor en forma del **problema de consulta N+1**, que esta práctica identifica y corrige con el patrón **DataLoader**.

2. Objetivos

1. Diseñar e implementar un esquema GraphQL (**Schema-First**) que unifique clientes y facturas en un solo grafo.
2. Identificar el problema de rendimiento **N+1** mediante los logs de consola de la base de datos simulada.
3. Mitigar el problema con el patrón **DataLoader** (agrupamiento por lotes y caché por petición).

3. Entorno y herramientas

Herramienta	Versión	Uso
Node.js	v24 (mínimo v18)	Entorno de ejecución con ES Modules (<code>"type": "module"</code>)
@apollo/server	5.5.1	Servidor GraphQL y Apollo Sandbox en <code>http://localhost:4000/</code>
graphql	16.14.2	Implementación de referencia de GraphQL para JavaScript

Herramienta	Versión	Uso
dataloader	2.2.3	Agrupamiento por lotes (batching) y caché por petición
Visual Studio Code	—	Editor y terminal para observar los logs [DB READ]

4. Estructura del proyecto

Archivo	Contenido
schema.graphql	Contrato SDL: tipos <code>Cliente</code> , <code>Factura</code> , enum <code>EstadoFactura</code> y la query <code>clientes</code>
database.js	Mock DB en memoria con 5 clientes, 6 facturas y latencia simulada de 200 ms por consulta
resolvers.naive.js	Solución ingenua : una consulta de facturas por cada cliente (N+1)
resolvers.js	Solución optimizada : <code>Cliente.facturas</code> usa <code>loaders.facturasPorCliente.load()</code>
dataloaders.js	Función por lotes (Batch Function) y fábrica <code>createLoaders()</code>
server.js	Carga el esquema, elige los resolvers según el modo y crea los loaders por petición

5. Diseño del esquema (Schema-First)

Primero se escribió el contrato de la API en SDL, antes que el código. El esquema define los tipos `Cliente` y `Factura`, un `enum` cerrado para el estado de la factura y la relación jerárquica `facturas: [Factura!]!`. Esa firma garantiza que la lista nunca es nula: un cliente sin facturas devuelve una lista vacía. La query `clientes` es el punto de entrada del grafo.

6. Identificación del problema N+1

En la versión ingenua, GraphQL ejecuta el resolver `Cliente.facturas` una vez por cada cliente que devuelve `Query.clientes`. Cada ejecución consulta la base de datos por separado y no sabe que las demás existen:

```
facturas: async (parent) => await db.fetchFacturasByClienteId(parent.id)
```

Así, una sola petición produce **1 consulta de clientes + N consultas de facturas**. Con los 5 clientes del mock se imprimen **6 consultas**; con 1.000 clientes serían 1.001.

7. Solución con el patrón DataLoader

DataLoader se ubica entre los resolvers y la base de datos y aporta dos mecanismos:

- **Agrupamiento por lotes (batching):** `.load(id)` no consulta la base de datos, solo registra la llave. Al terminar el tick actual del event loop, DataLoader junta todas las llaves y llama **una sola vez** a la función por lotes, que ejecuta `WHERE cliente_id IN (...)`.
- **Caché por petición:** si el mismo ID se solicita dos veces dentro de una misma petición, se consulta una sola vez.

La implementación se hizo en tres pasos:

- Función por lotes** en `dataloaders.js`: recibe el arreglo de `clienteIds` y devuelve un resultado por llave, **en el mismo orden**, que es el contrato de DataLoader. El cliente `c-5` no tiene facturas y recibe `[]`.
- Resolver optimizado** en `resolvers.js`: `Cliente.facturas` llama a `loaders.facturasPorCliente.load(parent.id)`.
- Loader por petición** en `server.js`: `context: async () => ({ loaders: createLoaders() })`. Crear un loader nuevo en cada petición evita que la caché se comparta entre usuarios. Un loader global podría devolver a un usuario datos de otro y servir datos desactualizados.

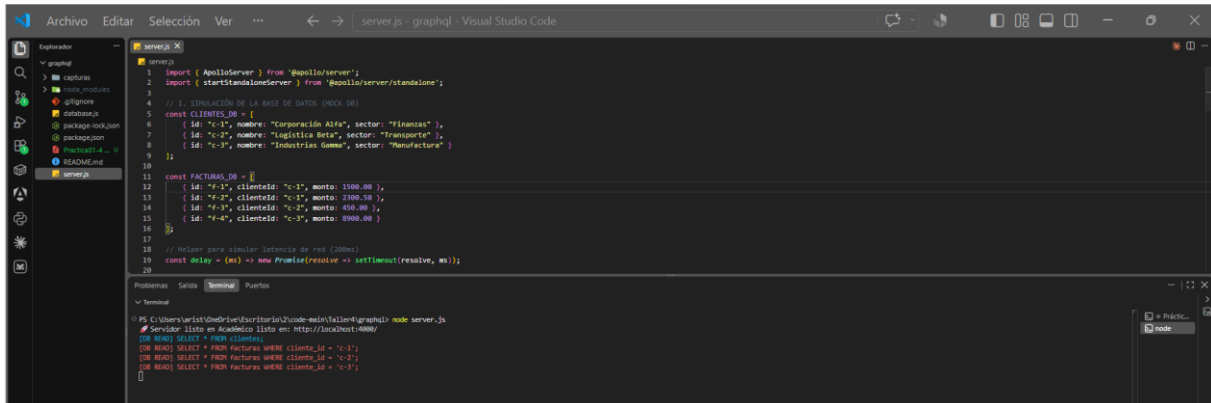
8. Resultados

Escenario	Comando	Consultas por petición
Código base (3 clientes)	<code>node server.js</code>	4 → 1 de clientes + 3 de facturas
Solución ingenua (5 clientes)	<code>npm run start:ingenuo</code>	6 → 1 de clientes + 5 de facturas (N+1)
Solución con DataLoader	<code>npm start</code>	2 → 1 de clientes + 1 en lote de facturas

Número exacto de consultas impresas tras aplicar DataLoader: 2. El número queda constante sin importar cuántos clientes haya. En esta simulación ambas versiones responden en unos 0,42 s, porque Apollo ejecuta en paralelo los resolvers de una lista y las esperas de 200 ms se solapan. La mejora real está en la **carga sobre la base de datos**: menos conexiones, menos CPU y menos viajes de red, que es lo que limita la escalabilidad cuando hay muchos usuarios a la vez.

Captura de Pantallas Funcionales

Pantalla 1 – Código base: problema N+1 con 3 clientes. Se ejecuta el `server.js` entregado por el docente, con los datos embebidos en el mismo archivo. Al lanzar la query del tablero, la terminal imprime 1 consulta a `clientes` (celeste) y 3 consultas individuales a `facturas` (rojo): 4 consultas en total para 3 clientes.



```

server.js
1 import { ApolloServer } from '@apollo/server';
2 import { startStandaloneServer } from '@apollo/server/standalone';
3
4 // 1. Definición de la base de datos (mock DB)
5 const CLIENTES_DB = [
6   { id: 'c-1', nombre: 'Corporación Alfa', sector: 'Finanzas' },
7   { id: 'c-2', nombre: 'Logística Beta', sector: 'Transporte' },
8   { id: 'c-3', nombre: 'Industrias Gamma', sector: 'Manufactura' }
9 ];
10
11 const FACTURAS_DB = [
12   { id: 'f-1', clienteId: 'c-1', monto: 1500.00 },
13   { id: 'f-2', clienteId: 'c-1', monto: 2300.00 },
14   { id: 'f-3', clienteId: 'c-2', monto: 450.00 },
15   { id: 'f-4', clienteId: 'c-3', monto: 8000.00 }
16 ];
17
18 // Helper para simular latencia de red (200ms)
19 const delay = (ms) => new Promise(resolve => setTimeout(resolve, ms));
20
Problemas Salida Terminal Puertos
C:\Users\javier\OneDrive\Escritorio\code-main\fallon4graphql> node server.js
Servidor listo en Académico listo en: http://localhost:4000/
[DB READ] SELECT * FROM clientes
[DB READ] SELECT * FROM facturas WHERE cliente_id = 'c-1';
[DB READ] SELECT * FROM facturas WHERE cliente_id = 'c-2';
[DB READ] SELECT * FROM facturas WHERE cliente_id = 'c-3';
  
```

Figura 1. Código base: 1 + 3 = 4 consultas a la base de datos.

Pantalla 2 – Apollo Sandbox ejecutando la query GetDashboardData. En <http://localhost:4000/> se ejecuta la query jerárquica que pide `nombre`, `sector` y el `monto` de las `facturas` de cada cliente. A la derecha, la respuesta JSON devuelve todo el grafo en una sola petición HTTP (estado 200, 426 ms). El panel Documentation muestra el campo `clientes: [Cliente!]!` publicado por el esquema.

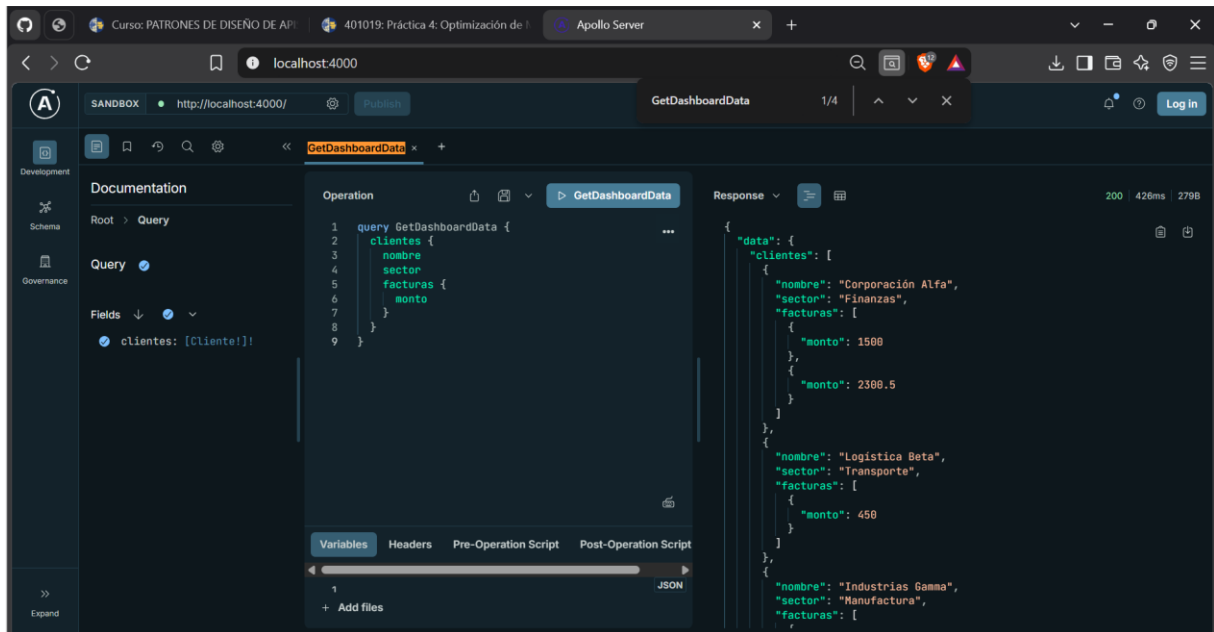


Figura 2. Una sola petición HTTP devuelve clientes y facturas.

Pantalla 3 – Solución ingenua con el esquema propio: 6 consultas. El servidor ya carga `schema.graphql` y usa `resolvers.naive.js` con la base de datos de 5 clientes (`database.js`). La terminal muestra 1 consulta `SELECT * FROM clientes` y 5 consultas `SELECT * FROM facturas WHERE cliente_id = ...`, una por cliente (c-1 a c-5). Este es el problema N+1 reproducido.

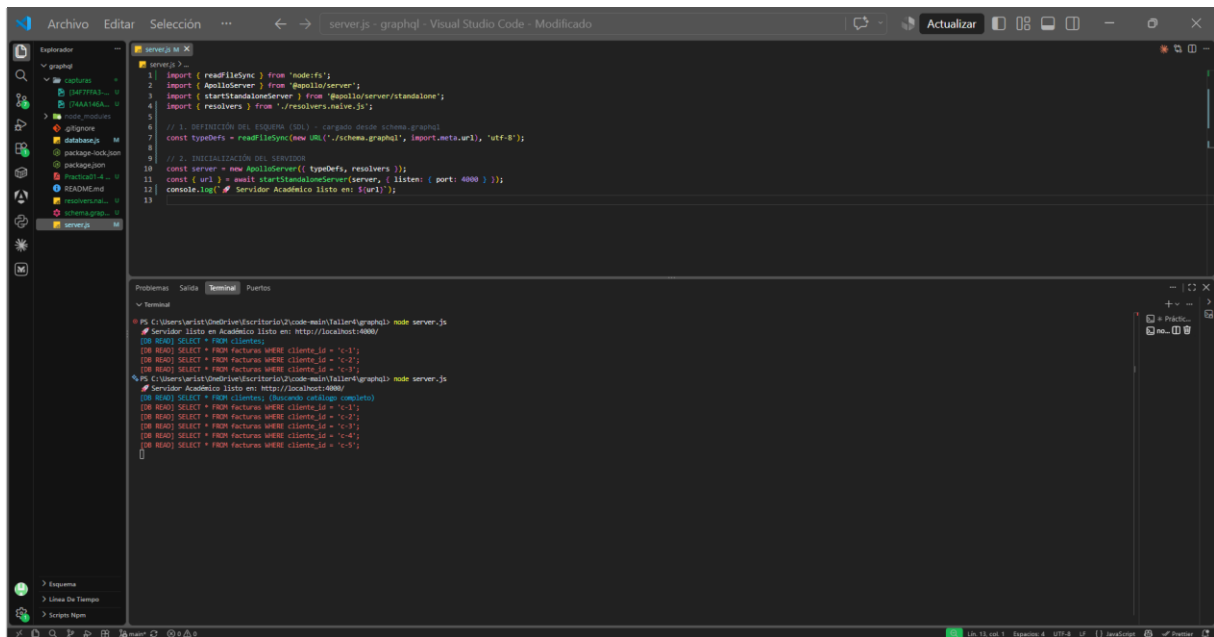


Figura 3. Solución ingenua: 1 + 5 = 6 consultas (problema N+1).

Pantalla 4 – Solución optimizada con DataLoader: 2 consultas. Con `npm start` el servidor arranca en modo OPTIMIZADO y crea los loaders dentro de `context`. Para la misma query, la terminal muestra solo 1 consulta de clientes y **1 consulta en lote** (`[DB BATCH READ] ... WHERE cliente_id IN ('c-1', 'c-2', 'c-3', 'c-4', 'c-5')`), en verde. Las 5 consultas individuales se redujeron a una.

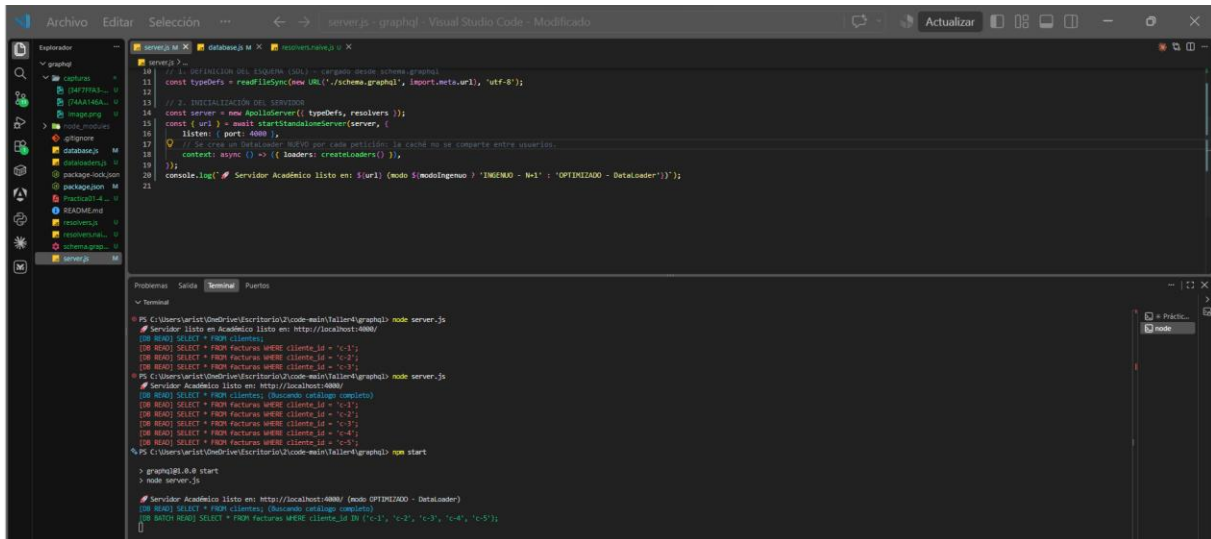


Figura 4. DataLoader: $1 + 1 = 2$ consultas, sin importar el número de clientes.

Código Fuente

Repositorio base del docente: <https://github.com/pmpvhr/graphql.git>

Repositorio con la solución: [enlace de GitHub del estudiante]

Ejecución: `npm install`, luego `npm start` (DataLoader, 2 consultas) o `npm run start:ingenueo` (N+1, 6 consultas). Abrir <http://localhost:4000/> y ejecutar la query:

```
query GetDashboardData {
  clientes {
    nombre
    sector
    facturas { monto estado }
  }
}
```

Archivo: `schema.graphql`

```
# LogiTech SaaS - Contrato de la API (Schema-First)
# Unifica clientes y sus facturas históricas en un solo grafo.

"Estado del ciclo de vida de una factura."
```

```
enum EstadoFactura {
  PAGADA
  PENDIENTE
  ANULADA
}

"Cliente corporativo de la plataforma."
type Cliente {
  id: ID!
  nombre: String!
  sector: String!
  "Facturas históricas del cliente (lista vacía si no tiene)."
  facturas: [Factura!]!
}

"Factura emitida a un cliente."
type Factura {
  id: ID!
  clienteId: ID!
  monto: Float!
  estado: EstadoFactura!
}

type Query {
  "Catálogo completo de clientes."
  clientes: [Cliente!]!
}
```

Archivo: *resolvers.naive.js*

```
import { db } from './database.js';

// RESOLUTORES - IMPLEMENTACIÓN INGENUA (provoca el problema N+1)
export const resolvers = {
  Query: {
    clientes: async () => await db.fetchAllClientes(),
  },
  Cliente: {
    facturas: async (parent) => {
      // El 'parent' es el cliente actual. GraphQL llama a esta función por CADA cliente
      devuelto,
      // generando 1 consulta de clientes + N consultas de facturas.
      return await db.fetchFacturasByClienteId(parent.id);
    }
  }
};
```

Archivo: *dataloaders.js*

```
import DataLoader from 'dataloader';
import { db } from './database.js';

/**
 * Batch Function: recibe TODAS las llaves (clienteIds) acumuladas durante un mismo tick
 * del event loop y las resuelve en un solo viaje a la base de datos.
 *
 * Contrato de DataLoader: debe devolver un arreglo del mismo tamaño y en el mismo orden
 * que las llaves recibidas (clientes sin facturas -> []).
 */
const batchFacturasPorCliente = async (clienteIds) => {
  return await db.fetchFacturasByClienteIdsBatch(clienteIds);
};

/**
 * Fábrica de loaders. Debe invocarse UNA VEZ POR PETICIÓN (en el `context` de Apollo)
 * para que la caché viva solo durante esa petición y no se compartan datos entre usuarios.
 */
export const createLoaders = () => ({
  facturasPorCliente: new DataLoader(batchFacturasPorCliente),
});
```

Archivo: *resolvers.js*

```
import { db } from './database.js';

// RESOLUTORES - IMPLEMENTACIÓN OPTIMIZADA (DataLoader: batching + caché por petición)
export const resolvers = {
  Query: {
    clientes: async () => await db.fetchAllClientes(),
  },
  Cliente: {
    facturas: (parent, _args, { loaders }) => {
      // En lugar de consultar la BD directamente, se encola la llave en el loader de la
      // petición.
      // DataLoader agrupa todas las llamadas .load() del mismo tick en UNA sola consulta
      // por lotes.
      return loaders.facturasPorCliente.load(parent.id);
    }
  }
};
```

Archivo: server.js

```
import { readFileSync } from 'node:fs';
import { ApolloServer } from '@apollo/server';
import { startStandaloneServer } from '@apollo/server/standalone';
import { createLoaders } from './dataloaders.js';

// Modo de ejecución: `node server.js` (optimizado) o `node server.js --ingenuo` (N+1)
const modoIngenuo = process.argv.includes('--ingenuo');
const { resolvers } = await import(modoIngenuo ? './resolvers.naive.js' : './resolvers.js');

// 1. DEFINICIÓN DEL ESQUEMA (SDL) - cargado desde schema.graphql
const typeDefs = readFileSync(new URL('./schema.graphql', import.meta.url), 'utf-8');

// 2. INICIALIZACIÓN DEL SERVIDOR
const server = new ApolloServer({ typeDefs, resolvers });
const { url } = await startStandaloneServer(server, {
  listen: { port: 4000 },
  // Se crea un DataLoader NUEVO por cada petición: la caché no se comparte entre usuarios.
  context: async () => ({ loaders: createLoaders() }),
});
console.log(`🚀 Servidor Académico listo en: ${url} (modo ${modoIngenuo ? 'INGENUO - N+1' : 'OPTIMIZADO - DataLoader'})`);
```

Archivo: database.js (fragmento – función por lotes)

```
/**
 * FUNCIÓN OPTIMIZADA PARA EL DATALOADER
 * Recupera las facturas de múltiples clientes utilizando una sola sentencia agrupada (Batch).
 * Los estudiantes deberán invocar esto dentro de su DataLoader.
 */
async fetchFacturasByClienteIdsBatch(clienteIds) {
  console.log(`\x1b[32m%\x1b[0m`, `[DB BATCH READ] SELECT * FROM facturas WHERE cliente_id IN (${clienteIds.map(id => `${id}`).join(', ')});`);
  await delay(200); // Mismo delay, pero trae la información de TODOS los ids provistos de un solo golpe.

  // Retorna un arreglo mapeado donde cada índice corresponde estrictamente a las llaves solicitadas (Restricción de DataLoader)
  return clienteIds.map(id =>
    FACTURAS_RECORDS.filter(factura => factura.clienteId === id)
  );
}
```

Conclusiones

1. GraphQL y DataLoader resuelven problemas distintos y se complementan. GraphQL elimina el under-fetching del **cliente**: las 1 + N peticiones HTTP que exigía REST se convierten en una sola query jerárquica definida por un contrato Schema-First. DataLoader elimina el N+1 del **servidor**: las consultas a la base de datos pasan de N + 1 (6 en la práctica) a un número constante (2), sin importar cuántos clientes haya.
2. El lugar donde se crea el DataLoader es una decisión de arquitectura, no un detalle de implementación. Instanciarlo dentro de **context** crea una caché nueva en cada petición: los

datos quedan aislados entre usuarios y no se sirven resultados viejos. Además, la función por lotes debe respetar el contrato de devolver un resultado por llave y en el mismo orden (incluido [] para clientes sin facturas). Si no lo hace, las facturas se asignan al cliente equivocado.

Bibliografía

Apollo Graph Inc. (2025). *Apollo Server documentation*.

<https://www.apollographql.com/docs/apollo-server>

GraphQL Foundation. (2021). *GraphQL specification* (October 2021 ed.).

<https://spec.graphql.org/October2021/>

GraphQL Foundation. (s. f.). *DataLoader* [Software]. GitHub.

<https://github.com/graphql/dataloader>

GraphQL Foundation. (s. f.). *Learn GraphQL: Performance*.

<https://graphql.org/learn/performance/>

OpenJS Foundation. (s. f.). *The Node.js event loop*. <https://nodejs.org/en/learn/asynchronous-work/event-loop-timers-and-nexttick>

Porcello, E., & Banks, A. (2018). *Learning GraphQL: Declarative data fetching for modern web apps*. O'Reilly Media.

pmpvhr. (2026). *graphql: Código base de la práctica GraphQL y DataLoader* [Repositorio]. GitHub.

<https://github.com/pmpvhr/graphql>